Aim: To implement a Decision Tree algorithm using Python

Software Required

- Python 3.x
- Jupyter Notebook
- Libraries: pandas, numpy, scikit-learn, matplotlib

Q1. What are Decision Trees?

Decision Trees are a **supervised machine learning algorithm** used for both **classification and regression** tasks. They work by recursively splitting the data based on feature values to form a tree-like structure where:

- **Internal nodes** represent decision rules based on features
- **Branches** represent outcomes of those decisions
- **Leaf nodes** represent final predictions (class labels or continuous values)

Q2. Explain different types of Decision Trees.

Type	Description	Use Case
Classification Trees	Predict categorical/discrete classes	Email spam detection, disease diagnosis
Regression Trees	Predict continuous/numerical values	House price prediction, sales forecasting
CART (Classification and Regression Trees)	Can handle both classification and regression	General-purpose tasks
ID3 (Iterative Dichotomiser 3)	Uses Information Gain with categorical data	Simple classification problems
C4.5 (Successor to ID3)	Handles both categorical and numerical data	Practical applications with mixed data
C5.0	Improved version of C4.5, more memory efficient	Commercial applications

Q3. List Advantages of Decision Trees.

- Easy to Understand & Interpret: Visual representation makes it transparent
- Requires Little Data Pre-processing: No need for feature scaling or normalization
- Handles Both Numerical & Categorical Data
- Non-Parametric: No assumptions about data distribution
- Feature Importance: Naturally shows which features are most important

- Handles Non-Linear Relationships
- Fast Prediction: Once trained, predictions are O(log n)

Q4 List Disadvantages of Decision Trees.

- Overfitting: Tend to create overly complex trees that don't generalize well
- High Variance: Small changes in data can create very different trees
- Biased with Imbalanced Data: May create biased trees favouring dominant classes
- Not Suitable for XOR-type Problems: Struggle with certain logical operations
- Instability: Sensitive to minor variations in training data
- Greedy Algorithm: Makes locally optimal decisions that may not be globally optimal

Q5. Draw the Decision Tree using ID3 Induction Method.

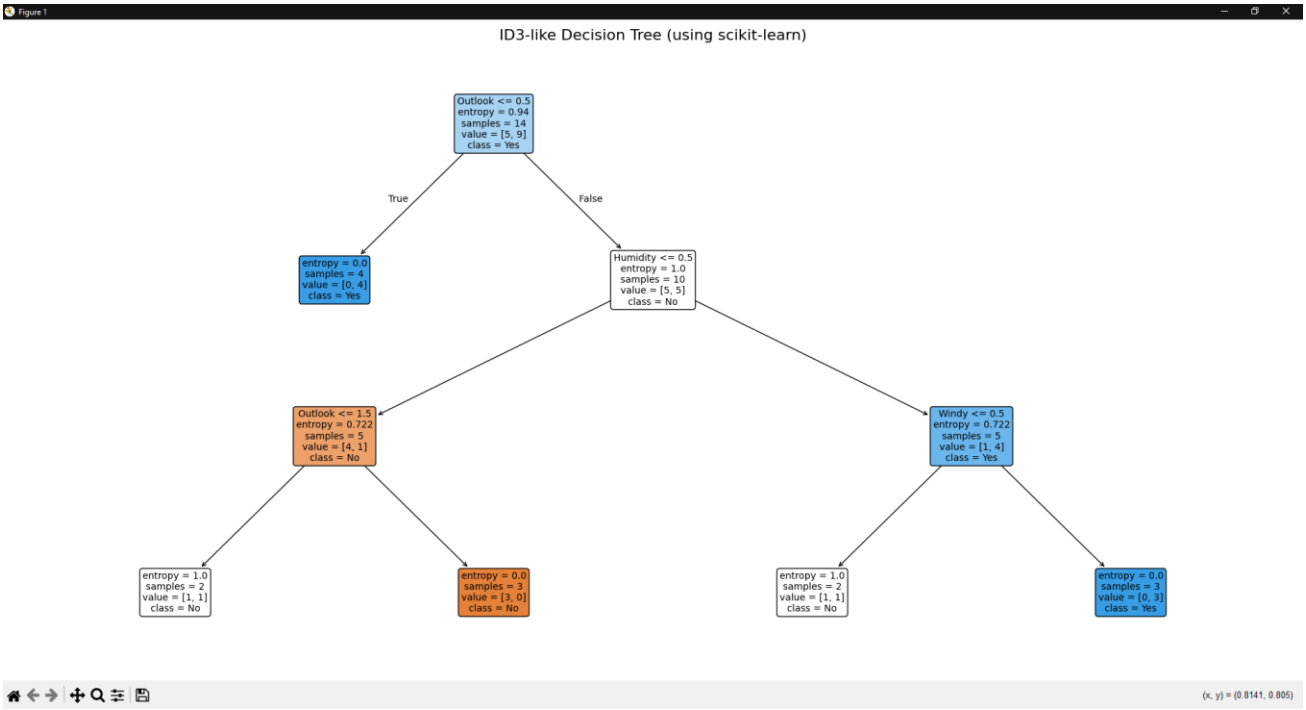
Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.preprocessing import LabelEncoder

# Define the dataset
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast',
                'Sunny', 'Sunny', 'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool',
                    'Mild', 'Cool', 'Mild', 'Mild', 'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal',
                'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'High'],
    'Windy': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong',
             'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes',
                   'Yes', 'Yes', 'Yes', 'Yes', 'No']
}

# Create Data Frame
df = pd.DataFrame(data)
print("Dataset:")
print(df)
print("\n" + "="*50)

# Features and target
features = ['Outlook', 'Temperature', 'Humidity', 'Windy']
target = 'PlayTennis'

# Prepare data for sklearn
X = df[features]
y = df[target]

# Encode categorical variables
le = LabelEncoder()
X_encoded = X.copy()
```

```
for col in X.columns:
    X_encoded[col] = le.fit_transform(X[col])

y_encoded = le.fit_transform(y)

print("\nEncoded Features:")
print(X_encoded.head())
print(f"\nEncoded Target (0=No, 1=Yes): {y_encoded}")

# Build decision tree with entropy criterion (similar to ID3)
dt_clf = DecisionTreeClassifier(
    criterion='entropy',  # This makes it similar to ID3
    max_depth=3,
    random_state=42
)

dt_clf.fit(X_encoded, y_encoded)

# Visualize the tree
plt.figure(figsize=(15, 10))
plot_tree(
    dt_clf,
    feature_names=features,
    class_names=['No', 'Yes'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("ID3-like Decision Tree (using scikit-learn)", fontsize=16)
plt.tight_layout()
plt.show()

# Print tree rules
print("\n" + "="*50)
print("Decision Tree Rules:")
print("="*50)
tree_rules = export_text(
    dt_clf,
    feature_names=features,
    spacing=3,
    decimals=2
)
print(tree_rules)

# Feature importance
print("\n" + "="*50)
print("Feature Importance:")
print("="*50)
for feature, importance in zip(features, dt_clf.feature_importances_):
    print(f"{feature}: {importance:.3f}")

# Make predictions
print("\n" + "="*50)
print("Making Predictions:")
print("="*50)
```

```
# Example test cases
test_cases = [
    {'Outlook': 'Sunny', 'Temperature': 'Hot', 'Humidity': 'High', 'Windy': 'Weak'},
    {'Outlook': 'Rain', 'Temperature': 'Cool', 'Humidity': 'Normal', 'Windy': 'Strong'},
    {'Outlook': 'Overcast', 'Temperature': 'Mild', 'Humidity': 'Normal', 'Windy': 'Weak'}
]

for i, test_case in enumerate(test_cases):
    # Encode the test case
    encoded_case = []
    for feature in features:
        # Get the label encoder for this feature
        le = LabelEncoder()
        le.fit(df[feature])  # Fit with original data
        encoded_value = le.transform([test_case[feature]])[0]
        encoded_case.append(encoded_value)

    prediction = dt_clf.predict([encoded_case])[0]
    probability = dt_clf.predict_proba([encoded_case])[0]

    print(f"\nTest Case {i+1}:")
    print(f"  Features: {test_case}")
    print(f"  Prediction: {'Yes (Play Tennis)' if prediction == 1 else 'No (Don\'t Play
Tennis)'}")
    print(f"  Confidence: {probability[prediction]*100:.1f}%")
    print(f"  Probabilities: [No: {probability[0]:.2%}, Yes: {probability[1]:.2%}]")
```

Output:



```
spectre@ghouled-gadget MINGW64 /d/degree/SEM 6/DMBI/Experiments/EXP5 (master)
$ py id3.py
Dataset:
     Outlook Temperature Humidity   Windy PlayTennis
0      Sunny         Hot     High    Weak         No
1      Sunny         Hot     High  Strong         No
2   Overcast         Hot     High    Weak        Yes
3       Rain        Mild     High    Weak        Yes
4       Rain        Cool   Normal    Weak        Yes
5       Rain        Cool   Normal  Strong         No
6   Overcast        Cool   Normal  Strong        Yes
7      Sunny        Mild     High    Weak         No
8      Sunny        Cool   Normal    Weak        Yes
9       Rain        Mild   Normal    Weak        Yes
10     Sunny        Mild   Normal  Strong        Yes
11  Overcast        Mild     High  Strong        Yes
12  Overcast         Hot   Normal    Weak        Yes
13      Rain        Mild     High  Strong         No

==================================================

Encoded Features:
   Outlook  Temperature  Humidity  Windy
0        2            1         0      1
1        2            1         0      0
2        0            1         0      1
3        1            2         0      1
4        1            0         1      1

Encoded Target (0=No, 1=Yes): [0 0 1 1 1 0 1 0 1 1 1 1 1 0]
```

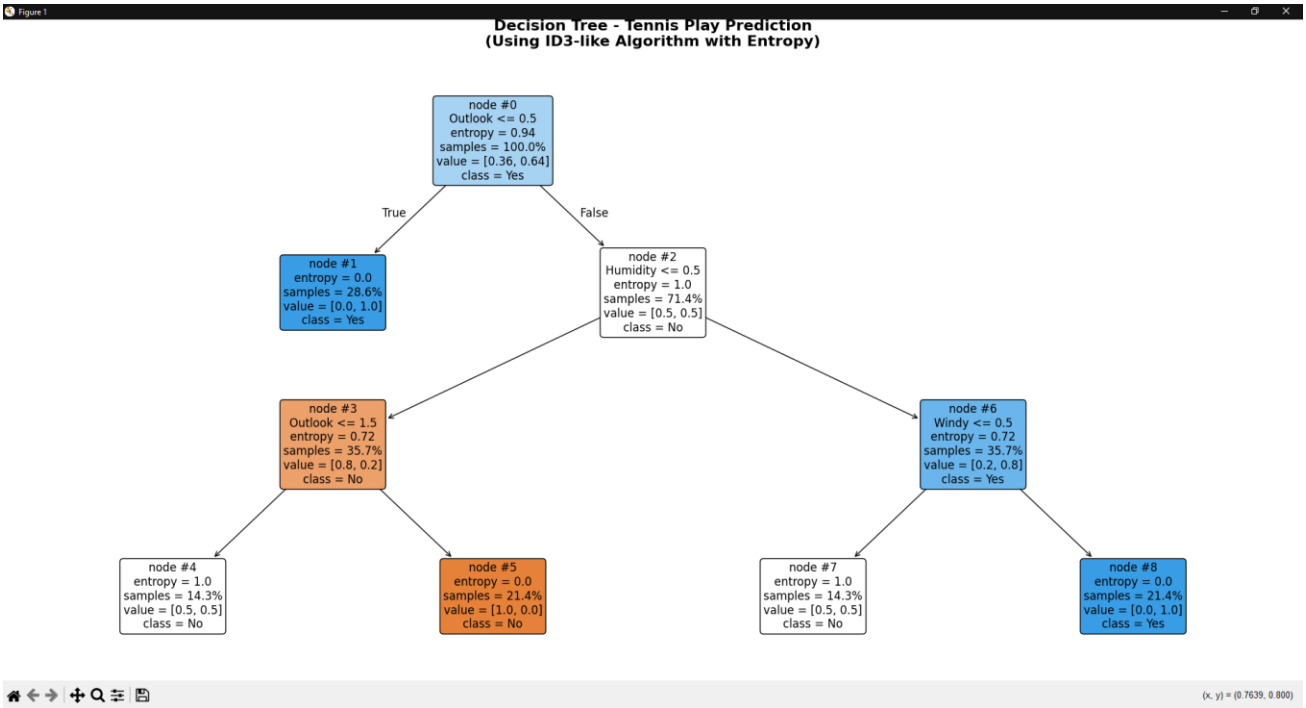
Q6. Create the Decision Tree using Naïve bayes Algorithm Method.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder

data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast',
                'Sunny', 'Sunny', 'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool',
                    'Mild', 'Cool', 'Mild', 'Mild', 'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal',
                'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'High'],
    'Windy': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong',
             'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes',
                   'Yes', 'Yes', 'Yes', 'Yes', 'No']}

df = pd.DataFrame(data)
features = ['Outlook', 'Temperature', 'Humidity', 'Windy']
target = 'PlayTennis'

le = LabelEncoder()
X_encoded = df[features].copy()
```

```
for col in features:
    le.fit(df[col])
    X_encoded[col] = le.transform(df[col])
y_encoded = le.fit_transform(df[target])

dt_clf = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=3,
    random_state=42
)
dt_clf.fit(X_encoded, y_encoded)
plt.figure(figsize=(20, 12))
plot_tree(
    dt_clf,
    feature_names=features,
    class_names=['No', 'Yes'],
    filled=True,
    rounded=True,
    fontsize=12,
    proportion=True,
    impurity=True,
    node_ids=True,
    precision=2
)
plt.title("Decision Tree - Tennis Play Prediction\n(Using ID3-like Algorithm with
Entropy)", fontsize=16, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()
```

Output:



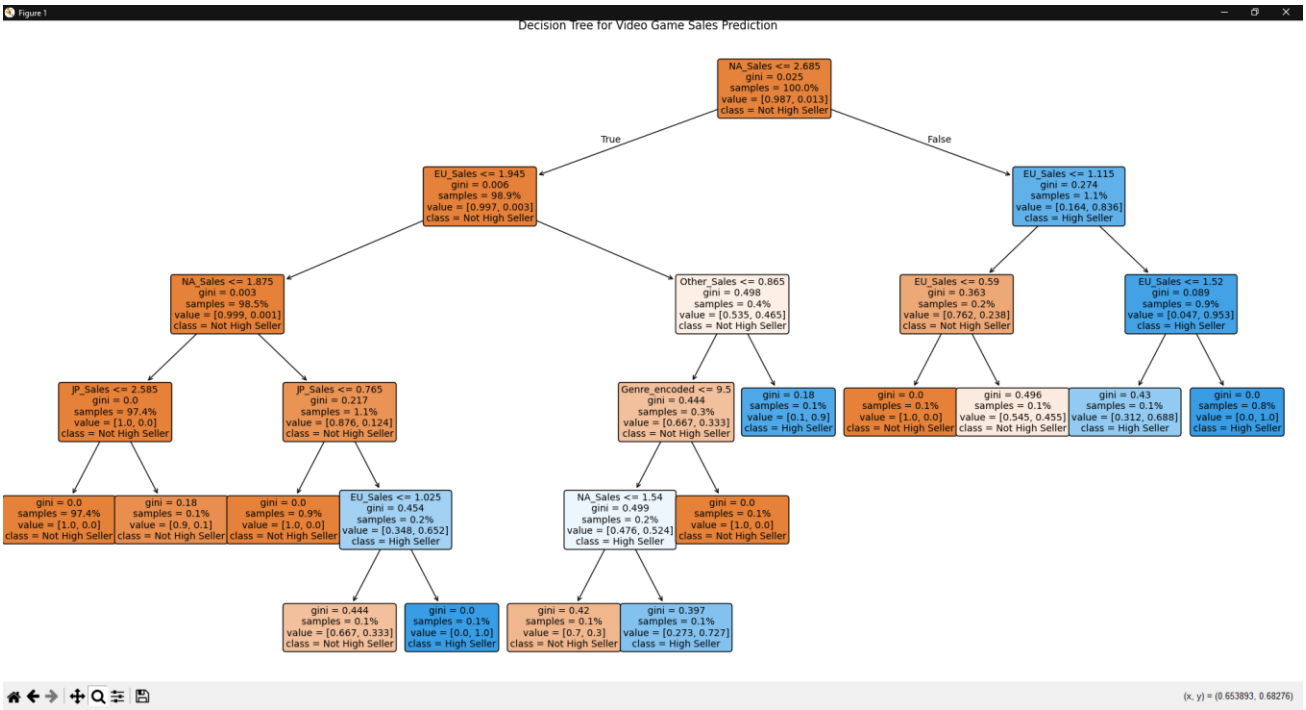
Q7. Write the Python Code for Decision Tree.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

df = pd.read_csv('vgsales.csv')

df['High_Seller'] = df['Global_Sales'].apply(lambda x: 1 if x > 5 else 0)

label_encoders = {}
categorical_cols = ['Platform', 'Genre', 'Publisher']

for col in categorical_cols:
    le = LabelEncoder()
    df[col + '_encoded'] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le

features = ['Year', 'Platform_encoded', 'Genre_encoded', 'Publisher_encoded',
            'NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales']

df_clean = df.dropna(subset=features + ['High_Seller'])

X = df_clean[features]
y = df_clean['High_Seller']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

dt_classifier = DecisionTreeClassifier(
    max_depth=5,
    min_samples_split=20,
    min_samples_leaf=10,
    random_state=42
)

dt_classifier.fit(X_train, y_train)

y_pred = dt_classifier.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

feature_importance = pd.DataFrame({
    'feature': features,
    'importance': dt_classifier.feature_importances_
}).sort_values('importance', ascending=False)

print("\nFeature Importance:")
print(feature_importance)
```

```
plt.figure(figsize=(20, 10))
plot_tree(
    dt_classifier,
    feature_names=features,
    class_names=['Not High Seller', 'High Seller'],
    filled=True,
    rounded=True,
    proportion=True,
    fontsize=10
)
plt.title("Decision Tree for Video Game Sales Prediction")
plt.tight_layout()
plt.show()

print("\nDecision Tree Rules (first 5 levels):")
tree_rules = export_text(
    dt_classifier,
    feature_names=features,
    max_depth=5
)
print(tree_rules)
```

Output:



Conclusion:

Decision Trees create hierarchical decision rules that make predictions through intuitive, tree-structured models.